



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-6276 stale custom cookie host causes cookie leak

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-13
Version	1.0
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-6276 stale custom cookie host causes cookie leak - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-13 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

1. Executive Summary

CVE-2026-6276 is a libcurl cookie-isolation flaw that can leak cookies across requests when an application reuses the same easy handle after first setting a custom `Host:` header and then issuing a later request without that custom header. curl upstream states that stale host-state from the first request can be reused for cookie selection and `Set-Cookie` attribution on the second request, causing cookies intended for one host to be sent to another host instead. In practical terms, this is an origin-confusion problem inside application-managed HTTP workflows rather than a generic browser-style cookie bug.

The exposure is narrower than many network-facing curl advisories. Upstream explicitly rates the issue Low, notes that setting a custom `Host:` header is mostly a debugging or specialized application behavior, and emphasizes that HTTPS virtual-host abuse is naturally constrained because a forged `Host:` header alone does not fix the SNI mismatch required for a clean TLS virtual-host pivot. curl also states that the command-line `curl` tool is not affected. The real risk therefore sits with applications embedding libcurl and manually controlling HTTP headers while also reusing handles and enabling cookies.

For defenders, the issue still matters because libcurl is deeply embedded in package downloaders, API clients, orchestration platforms, appliances, agents, data movers, and bespoke internal services. If a vulnerable application uses custom `Host:` values for routing, testing, tenancy, or reverse-proxy integration, the stale `cookiehost` state can break origin boundaries and disclose authenticated session material to an unintended peer. The consequence is usually information disclosure and trust-boundary erosion rather than remote code execution, but the downstream impact can still be significant where leaked cookies grant access to administrative or multi-tenant application contexts.

The immediate defensive priorities are to upgrade libcurl to 8.20.0 or a verified vendor-backported build, identify applications that set custom `Host:` headers through libcurl APIs, and review any cookie-bearing libcurl workflows that reuse easy handles across multiple origins or virtual-host contexts. Detection and hunting should focus on exposure discovery and anomalous host/cookie combinations in application, proxy, and reverse-proxy telemetry rather than on classic IOC matching. Reviewed public sources did not provide authoritative evidence of broad in-the-wild exploitation or KEV inclusion at assessment time.

2. Intelligence Requirement

This report addresses the following intelligence requirement: what does CVE-2026-6276 mean for defenders using libcurl in custom-header workflows, which environments are materially exposed, how should severity be interpreted, and what remediation, detection, and hunting actions should be prioritized now?

Question	Assessment
What is the flaw?	Reused libcurl easy handles can retain stale custom-host state and misapply cookies to a later request made without the original custom `Host:` header.
What is the primary impact?	Cookie disclosure and incorrect cookie attribution across request-origin boundaries.
Which products are affected?	libcurl-based applications using cookies, custom `Host:` headers, and easy-handle reuse.
Is the `curl` CLI affected?	No. curl upstream explicitly states the command-line tool is not affected.
Which versions are affected?	curl/libcurl 7.71.0 through 8.19.0 inclusive, according to the curl advisory and curl JSON metadata.
What fixes the issue?	curl/libcurl 8.20.0 or vendor backports carrying the same remediation.
Why is vendor severity Low?	The trigger is specialized, usually associated with debugging or uncommon application logic, and HTTPS host pivoting is partly constrained by SNI requirements.

Why should defenders still care?	libcurl is widely embedded, and leaked cookies can still cross trust boundaries inside internal services, proxies, or multi-tenant application paths.
Is broad exploitation confirmed?	Reviewed public sources did not identify authoritative evidence of widespread in-the-wild exploitation at assessment time.
Which environments deserve highest priority?	Embedded libcurl applications that manually set `Host:` headers, maintain cookie jars, and reuse handles across different logical destinations or tenants.

3. Source Review and Confidence

Source	Contribution	Confidence
curl security advisory (CVE-2026-6276.html)	Canonical flaw description, severity, affected versions, solution, timeline, and explicit note that the curl CLI is not affected	High
curl machine-readable advisory JSON	Structured confirmation of version range, fix commit, introducing commit, CWE, and publication timing	High
curl patch commit `3a19987a87f393d9394fe5acc7643f6c263c92db`	Concrete engineering details showing `cookiehost` moved into per-request state and reset appropriately	High
HackerOne disclosure page metadata	Summary language confirming stale `data->state.aptr.cookiehost` behavior and cookie-selection / `Set-Cookie` attribution mechanics	Medium
NVD CVE record	Public enrichment, affected CPE range, and ecosystem tracking references	Medium
Openwall oss-security republication	Confirms coordinated disclosure language and mirrors the upstream advisory for timeline corroboration	High
Microsoft Security Update Guide syndication entry	Workflow trigger confirming publication visibility in monitored intake feeds	Medium

Confidence is high for the core technical assessment because the upstream curl advisory, curl JSON metadata, and public fix commit align on the vulnerability trigger, affected versions, and remediation path. Confidence is lower for exploitation prevalence because the reviewed public source set did not include incident-response reporting, KEV inclusion, or widespread campaign telemetry.

One analyst caveat matters: public severity normalization is inconsistent. curl upstream labels the issue Low and frames it as a specialized origin-validation bug in libcurl-based applications, while NVD currently publishes a CVSS 3.1 score of 7.5 High with an availability-focused vector that does not cleanly match the vendor's cookie-leak narrative. Defenders should privilege the vendor's technical scoping for workflow exposure and use NVD mainly for ecosystem cross-referencing rather than for business-priority decisions in isolation.

4.1 Vulnerability metadata

Field	Value
-------	-------

CVE	CVE-2026-6276
Product family	curl / libcurl
Vulnerability class	Cross-request cookie leakage caused by stale custom-host state
Primary weakness	CWE-346 Origin Validation Error
Vendor severity	Low
Public scoring note	NVD currently scores the issue CVSS 3.1 7.5 High, which appears misaligned with the vendor-described information-disclosure mechanics
Affected versions	curl/libcurl 7.71.0 through 8.19.0 inclusive
Fixed version	curl/libcurl 8.20.0
Introduced by	curl commit `e15e51384a423be31318b3c9c7d612a1aae661fd`
Fixed by	curl commit `3a19987a87f393d9394fe5acc7643f6c263c92db`
Public disclosure	2026-04-29 upstream advisory publication; 2026-06-13 MSRC syndication entry reviewed by this workflow
Exploitation status	No authoritative broad in-the-wild exploitation confirmation identified during this review
Notable carve-out	The curl command-line tool is not affected

4.2 Flaw mechanics

The flaw exists because libcurl stored cookie-host information derived from a custom `Host:` header in state that could outlive the individual request. When an application performed a first HTTP request using a custom host value and then reused the same easy handle for a later request without that custom header, libcurl could continue using stale host information for both outbound cookie matching and inbound `Set-Cookie` processing.

The curl fix is technically instructive. The remediation commit moves `cookiehost` from broader easy-handle state into the `SingleRequest` structure and explicitly clears it when preparing a new request. curl's added regression test (`lib2504`) verifies the vulnerable sequence: one request sets `Host: victim.internal`, a later request on the same handle removes the custom header, and the library must not continue using the earlier host context for cookie handling.

This is not a generic cookie theft issue affecting every libcurl consumer. Exploitation requires all of the following conditions to overlap:

- The application embeds libcurl rather than only invoking the curl CLI.
- Cookies are enabled or relied upon.
- The application sets custom `Host:` headers.
- The same easy handle is reused across requests.
- The origin or trust boundary meaningfully changes between requests.

4.3 Exposure profile

Exposure pattern	Why it matters	Priority
Internal applications using libcurl with manual `CURLOPT_HTTPHEADER` / custom `Host:` values	Directly matches the trigger condition described by curl and the remediation commit	Immediate

Reverse-proxy, API-gateway, or multitenant service integrations that rely on custom host routing	Cookie scope confusion can cross tenant or administrative boundaries	Immediate
Test harnesses, health checkers, synthetic monitoring, or appliance software that reuse libcurl easy handles	These workflows often optimize for handle reuse and custom routing logic	High
Cookie-authenticated internal APIs contacted over cleartext HTTP in trusted zones	Reduces the TLS/SNI mitigating effect described by curl and can make host confusion easier to realize	High
HTTPS-only workflows that never set custom `Host:` and never reuse handles across origins	Materially reduces or removes the vendor-described preconditions	Reduced
Environments relying only on the `curl` command-line tool	Upstream states they are not affected by this CVE	Reduced

4.4 Defensive significance and ATT&CK framing

MITRE ATT&CK mapping is necessarily conditional because CVE-2026-6276 is a library-origin-boundary flaw rather than a full adversary playbook. The best-fit mappings should be treated as defensive framing for telemetry and impact discussion, not as proof of a specific actor technique chain.

ATT&CK ID	Technique	Rationale
T1071.001	Application Layer Protocol: Web Protocols	The vulnerable behavior occurs inside HTTP request handling and cookie exchange over web protocols.
T1539	Steal Web Session Cookie	Successful exploitation can disclose cookies associated with one host context to another unintended destination or request context.

The practical meaning is clearer than the ATT&CK labels: any internal or external application that uses libcurl to override host identity while reusing stateful handles should be treated as a potential cookie-boundary failure until patched or reviewed.

5. Threat Context

CVE-2026-6276 is best understood as a secure-development and application-integration issue, not an internet-wide one-click exploit. The flaw sits at the intersection of state reuse, cookie handling, and application-specific routing logic. That makes it especially relevant for product teams, appliance vendors, and internal platform owners who embed libcurl and may not realize how much transfer state survives between requests on a reused easy handle.

The vendor's low-severity stance is justified for many environments. Setting a custom `Host:` header is atypical in normal application traffic, and HTTPS host pivots are constrained because a changed HTTP host header alone does not fully redirect TLS virtual-host semantics without matching SNI and certificate behavior. Many enterprises also use modern service patterns where origin identity is encoded directly in URLs or higher-level client libraries rather than by manually overriding libcurl headers.

Defenders should nevertheless avoid dismissing the issue outright. In internal service meshes, debugging proxies, integration adapters, and multi-tenant reverse-proxy workflows, developers sometimes use custom `Host:` values specifically to steer traffic while keeping the transport destination unchanged. When cookies carry administrative or cross-tenant session meaning, a stale host-state bug can create data disclosure conditions that are subtle, hard to observe, and easy to miss in routine patch triage.

5.1 Representative abuse scenarios

Scenario	Description	Defensive meaning
Multi-tenant internal gateway using host-based routing	A libcurl-based client reuses a handle across tenant contexts and sends cookies scoped to a previous host identity into a later request	Review tenant isolation and session-cookie scope assumptions
Appliance or agent with synthetic monitoring logic	Monitoring code sets custom hosts for test routing and reuses handles across checks, accidentally carrying cookie state into the wrong destination	Patch bundled libcurl and audit internal health-check logic
Reverse-proxy integration over cleartext HTTP in trusted zones	Teams depend on custom `Host:` to steer requests while cookies authenticate to downstream apps, increasing the chance of cross-origin leakage	Treat custom-host plus cookies as a privileged design pattern requiring review
QA or debugging workflow promoted into production code	A once-temporary host-override path becomes permanent in an application that now handles live authenticated traffic	Search codebases for custom-header overrides and stateful handle reuse

5.2 Exploitation-status assessment

Reviewed public sources did not identify authoritative evidence of widespread exploitation, KEV listing, or active campaign reporting. The public record instead emphasizes disclosure, patching, and regression-test coverage. That lowers concern for indiscriminate internet-scale abuse.

Exposure should therefore be prioritized based on application design and cookie sensitivity, not on media volume. If a vulnerable internal service uses host overrides to cross meaningful security boundaries, even a low-severity upstream issue can justify rapid remediation and targeted hunt activity. Conversely, estates that only use the curl CLI or that never combine cookies with custom host routing can safely assign lower urgency after validating those assumptions.

6. Detection and Hunting

Detection for CVE-2026-6276 should focus on exposure discovery, code-path review, and suspicious host/cookie combinations in telemetry. No stable attacker domains, malware hashes, or campaign-specific IOCs were identified in the reviewed public source set. This is therefore a vulnerability-management and architecture-hunting problem rather than a traditional IOC-led incident problem.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal	Coverage caveat
Installed curl/libcurl below fixed or backported level	Software inventory, SBOM, package manager telemetry	Identify endpoints and applications needing version review	Vendor backports may remediate without matching upstream version strings
Source code or binaries that reference `CURLOPT_HTTPHEADER`, `Host:`, or cookie APIs	Secure code review, SAST, repo grep, binary strings	Find applications likely to satisfy the custom-host + cookies preconditions	Code presence does not prove exploitable runtime behavior
Applications making HTTP requests with host-header overrides and cookie-bearing responses	Reverse-proxy logs, app telemetry, packet capture in controlled testing	Detect runtime patterns consistent with the vulnerable design	Most environments do not log header details by default

Reused application sessions contacting multiple logical host identities with the same process or service account	App logs, service tracing, EDR network/process correlation	Surface state reuse across trust boundaries	Hard to confirm without rich app-level telemetry
Unexpected `Set-Cookie` attribution or cookie forwarding to a later destination	Reverse proxies, QA harnesses, debug traces	Validate whether cross-request contamination is occurring	Often only feasible in lab, staging, or verbose debug mode

6.2 Sigma - custom Host usage in curl-style tooling for exposure review

```

title: Custom Host Header Usage in curl-Style Tooling Requiring CVE-2026-6276 Review
id: 6af4068d-c317-4d85-9db0-d1e93bc12f42
status: experimental
description: Detects command-line usage of custom Host headers in curl-style tooling. This is primarily useful for exposure discovery and validation because the curl CLI itself is not affected, but similar patterns can reveal operational dependence on host overrides that should be reviewed in libcurl-based applications.
author: Lost Boys Cyber
date: 2026-06-14
references:
  - https://curl.se/docs/CVE-2026-6276.html
  - https://curl.se/docs/CVE-2026-6276.json
logsource:
  category: process_creation
  product: windows
detection:
  image:
    Image|endswith:
      - '\curl.exe'
  host_override:
    CommandLine|contains:
      - '-H "Host:'
      - '-H "Host:"
      - '--header "Host:'
      - '--header "Host:"
  condition: image and host_override
falsepositives:
  - Legitimate troubleshooting, QA, or infrastructure validation
  - Security-team reproduction of host-routing behaviors
level: low
tags:
  - attack.t1071.001

```

This analytic does not prove exploitation and should not be interpreted as direct detection of the libcurl flaw. Its value is identifying environments where teams operationally depend on host overrides and may also have adjacent libcurl implementations worth reviewing.

6.3 KQL - inventory curl and libcurl installations for remediation triage

```

DeviceTvmSoftwareInventory
| where SoftwareName has_any ("curl", "libcurl")
| project DeviceName, SoftwareName, SoftwareVersion, Vendor
| order by DeviceName asc, SoftwareName asc

```

6.4 KQL - hunt for code or script artifacts referencing custom Host and cookie handling

```

let lookback = 30d;
DeviceFileEvents
| where Timestamp > ago(lookback)
| where FileName in~ ("*.c", "*.cpp", "*.cc", "*.h", "*.py", "*.sh", "*.ps1", "*.go", "*.rs", "*.java", "*.cs")
| where FolderPath has_any ("src", "app", "scripts", "agent", "monitor", ".cargo", "workspace")
| where InitiatingProcessFileName !in~ ("System")
| project Timestamp, DeviceName, FileName, FolderPath, ActionType, InitiatingProcessFileName,

```

```
InitiatingProcessCommandLine
| order by Timestamp desc
```

Because many EDR schemas do not capture file-content searches directly, defenders should pair this with repository scanning or developer-environment grep for `CURLOPT\_HTTPHEADER`, `Host:`, `CURLOPT\_COOKIEFILE`, `CURLOPT\_COOKIEJAR`, and similar libcurl cookie controls.

6.5 SPL - proxy or application log hunt for cookie-bearing requests with host anomalies

```
index=proxy OR index=web
("Host:" OR cs_host=* OR http_host=*)
(cookie=* OR set_cookie=* OR http_cookie=*)
| eval observed_host=coalesce(cs_host,http_host,host_header)
| eval observed_dest=coalesce(dest_host,server_name,uri_host)
| where isnotnull(observed_host) AND isnotnull(observed_dest) AND observed_host!=observed_dest
| stats count values(uri) values(user_agent) values(src_ip) values(observed_dest) by observed_host,
app, process_name
| sort - count
```

6.6 Hunt questions

Hypothesis	What to prove or disprove
Some internal applications embed libcurl and override `Host:` for routing or testing	Search source repos, build manifests, and appliance bundles for libcurl plus custom-header logic
Cookie-bearing requests are being made across reused handles or shared sessions	Review application design, connection pooling, and debug traces for handle reuse with cookies enabled
Internal reverse-proxy or tenant-routing workflows depend on custom host identity	Validate whether cookies, tenant boundaries, or admin sessions share that same path
Severity appears low upstream but could still be high impact locally	Map whether any leaked cookies would grant privileged, cross-tenant, or administrative access

No stable IOC set was identified from reviewed public sources. Hunting should center on architecture, code paths, and session-boundary assumptions.

7. Recommendations

Priority	Owner	Action	Rationale
Immediate	Platform / application owners	Upgrade libcurl to 8.20.0 or a validated vendor-backported equivalent	Removes the stale `cookiehost` behavior at the library layer
Immediate	Engineering leads	Identify applications that set custom `Host:` headers while also using cookies and easy-handle reuse	Directly maps to the vendor-described trigger conditions
Immediate	Security engineering	Classify whether any affected cookies guard privileged, multi-tenant, or administrative application contexts	Business impact depends on the sensitivity of the leaked session material
High	Developers / SRE	Remove unnecessary custom `Host:` overrides and avoid reusing stateful easy handles across different logical origins	Reduces the architectural preconditions even before all patches land
High	QA / product security	Add regression tests mirroring	Prevents future reintroduction

		curl's `lib2504` sequence to internal libcurl consumers with custom routing logic	in product-specific wrappers or forks
High	SOC / detection engineering	Add inventory and host/cookie anomaly hunts in EDR, proxy, and application logging where available	Visibility for this issue is strongest through workflow discovery rather than IOC matching
Medium	Vulnerability management	Validate distribution and appliance backports before marking systems remediated or unaffected	Prevents false positives and false negatives where upstream version strings differ
Medium	Architecture review boards	Treat custom host overrides plus cookie state as a security-sensitive design pattern requiring explicit review	Helps catch similar origin-confusion issues beyond this specific CVE

Recommended execution order:

- Inventory curl/libcurl versions and identify embedded consumers that may hide library use from simple package scanning.
- Search for custom `Host:` handling plus cookies in application code, wrappers, and appliances.
- Patch to 8.20.0 or vendor-equivalent fixed builds, prioritizing multi-tenant, reverse-proxy, and authenticated internal-service workflows.
- Validate whether any affected applications reuse easy handles across different logical host identities.
- Add regression tests and telemetry hunts so future library updates or code changes do not silently reintroduce the same origin-boundary failure.

8. References

- curl Security Advisory, `CVE-2026-6276: stale custom cookie host causes cookie leak` - <https://curl.se/docs/CVE-2026-6276.html>
- curl machine-readable advisory JSON - <https://curl.se/docs/CVE-2026-6276.json>
- curl fix commit `3a19987a87f393d9394fe5acc7643f6c263c92db` - <https://github.com/curl/curl/commit/3a19987a87f393d9394fe5acc7643f6c263c92db>
- curl introducing commit `e15e51384a423be31318b3c9c7d612a1aae661fd` - <https://github.com/curl/curl/commit/e15e51384a423be31318b3c9c7d612a1aae661fd>
- NVD CVE record, `CVE-2026-6276` - <https://nvd.nist.gov/vuln/detail/CVE-2026-6276>
- Openwall oss-security advisory mirror - <http://www.openwall.com/lists/oss-security/2026/04/29/13>
- HackerOne disclosure page, `curl disclosed on HackerOne: CVE-2026-6276: stale custom cookie host causes cookie leak` - <https://hackerone.com/reports/3671818>
- Microsoft Security Update Guide syndication entry - <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-6276>